

The Reaction Wheel Pendulum

Modeling, differential equations, and feedback control — derived from first principles, for an engineer who will write every line of the simulation and the PID himself.

Prepared for R. Bafna · Instructor: Claude · Summer 2026 · Phases 0–8

THE ENTIRE IDEAL MACHINE – TWO LINES, THREE NUMBERS

$$J\ddot{\theta} = K_g \sin \theta - \tau \quad \text{arm}$$

$$I_w \ddot{\phi} = \tau \quad \text{wheel}$$

Gravity tips the arm over in proportion to $\sin \theta$. The one thing you control, motor torque τ , does two things at once: it spins the wheel up, and its Newton's-third-law reaction shoves the arm back. Everything in this course is inside these two lines.

CONTENTS

- | | |
|------------------------------------|-------------------------------------|
| §0 The machine and the mission | §5 Lens: it's a spring |
| §1 Roadmap: nine phases | §6 Lens: it's a momentum bank |
| §2 Rotational mechanics kit | §7 Lens: it's a saddle point |
| §3 The model, by forces & torques | §8 Differential equations from zero |
| §4 The model, by energy (Lagrange) | §9 Numerical integration & ode45 |

§10 The control chapter

§11 Swing-up (preview)

§12 Problem sets

§13 Reading list, glossary, notation

§14 Pathology reports from the sim lab

The machine and the mission

What you are building, why it is hard, and why it is the perfect first controls project.

A reaction wheel pendulum is a rod on a free pivot with a motor-driven flywheel mounted on it. You may not push the rod. You may not move the pivot (that would be the cart-pole — a different machine). The only thing you are allowed to do is put torque into the wheel. By Newton's third law, every bit of torque you put into the wheel comes back as an equal and opposite shove on the rod. Your mission: balance the rod upright using nothing but that recoil.

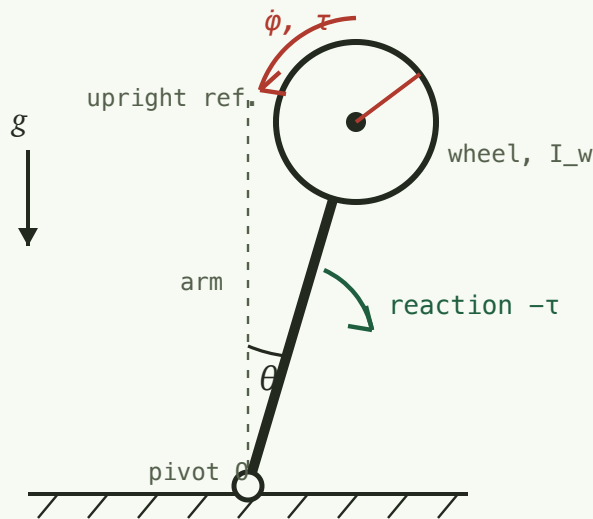


Fig. 1 – The machine. θ is the arm angle measured from upright ($\theta = 0$ is balanced), counter-clockwise positive. ϕ is the wheel's rotation in the world frame. Motor torque τ acts on the wheel; its reaction $-\tau$ acts on the arm. Spin the wheel counter-clockwise and the arm gets shoved clockwise.

This is the same physics family as spacecraft attitude control. Hubble and JWST point themselves with reaction wheels; ETH Zürich's *Cubli* balances a cube on its corner with three of them. When a Kepler reaction wheel failed, the telescope lost the ability to point — the wheel is the actuator.

Why it is the perfect first controls project

- **It is genuinely unstable.** Do nothing and it falls, exponentially, with a doubling time you will compute (about 82 ms for our example rig). Most beginner sims are stable systems wearing costumes; this one fights back.
- **It is underactuated.** Two degrees of freedom (arm angle, wheel angle), one motor. You cannot command both at once; you must exploit gravity. This is the defining constraint of legged robots, rockets, and acrobatic drones — in miniature.
- **Yet it fits in your head.** The ideal machine is three numbers and two short equations. You can hold the whole thing at once, which is exactly what a first project should allow.

The three numbers

J

PIVOT INERTIA

everything's
resistance to
rotation about the
pivot · kg·m²

K_g

GRAVITY STIFFNESS

how hard gravity
torques the arm per
radian of lean ·
N·m/rad

I_w

WHEEL INERTIA

the wheel's own spin
inertia – your
momentum bank · kg·m²

Plus one input, motor torque τ , and one honesty constraint, the torque limit $\tau_{\max}$. Friction adds two more numbers later; start without it.

The example rig

All worked numbers in these notes use this desktop-scale rig. Change the numbers to match whatever you end up modeling; the formulas are the point.

QUANTITY	SYMBOL	VALUE	WHERE IT COMES FROM
Arm mass	m_a	0.20 kg	uniform rod
Arm length	ℓ	0.15 m	pivot to tip; center of mass at $\ell/2 = 0.075$ m
Wheel mass	m_w	0.30 kg	disk mounted at the tip, $L = 0.15$ m
Wheel radius	r_w	0.05 m	solid disk
Pivot inertia	J	8.25×10^{-3} kg·m ²	$m_a \ell^2/3 + m_w L^2$ (§2)
Gravity stiffness	K_g	0.589 N·m/rad	$(m_a \cdot \ell/2 + m_w \cdot L) \cdot g$ (§3)
Wheel inertia	I_w	3.75×10^{-4} kg·m ²	$\frac{1}{2} \cdot m_w \cdot r_w^2$ (§2)
Torque limit	$\tau_{\max}$	0.10 N·m	assumed hobby-motor class

HOW TO USE THESE NOTES

- **Pencil before keyboard.** Every derivation here should be re-derived by you, on paper, before you believe it. The notes are the answer key, not the work.
- **Predict before you plot.** Before every simulation run, write down what you expect. The gap between prediction and plot is where the learning lives.
- **The pact:** the instructor writes no simulation, solver, or controller code. You bring derivations and results; I check them like a grader. I build the visualization front-end at the end (Phase 8).

Roadmap: nine phases

Each phase has a goal, a list of things to look up, and a checkpoint — a concrete artifact that proves you're done. Time estimates assume relaxed evenings.

PHASE 0 Get your bearings

~1 evening

Understand what the machine is and why it's hard. Watch it exist in the world. Set up a notebook environment (numpy, scipy, matplotlib).

LOOK UP: ETH Cubli video · reaction wheels on Hubble/JWST · "underactuated system" (just the definition)

CHECKPOINT: you can explain to a friend, without notes, why spinning the wheel moves the arm.

PHASE 1 Rotational mechanics kit

1–2 evenings

Torque, moment of inertia, the parallel-axis theorem, Newton's second law for rotation, third-law torque pairs, angular momentum. §2 of these notes.

LOOK UP: moment of inertia of a rod about its end and a disk about its center · parallel-axis theorem · angular momentum of a rigid body

CHECKPOINT: compute J , K_g , I_w for the example rig by hand and match the table in §0 (Problem Set 0, Q1).

PHASE 2 Model the machine

2–3 evenings

Derive the two equations of motion twice: once with forces and torques (§3), once with energy via Lagrange (§4). They must agree. This is the heart of the mechanical-engineering content.

LOOK UP: free-body diagram · Lagrangian mechanics (just the recipe: $T - V$, one equation per coordinate) · generalized force

CHECKPOINT: your hand derivation reproduces equations (1) and (3), and you can defend every sign in them with a physical argument.

PHASE 3 Differential equations from zero

3–5 evenings

What an ODE is, why exponentials are the atoms of dynamics, the characteristic-equation trick, oscillation vs. blow-up, damping ratio and natural frequency. §8. This is the math spine of the course.

LOOK UP: 3Blue1Brown differential equations series · Euler's formula $e^{i\omega t} = \cos \omega t + i \sin \omega t$ · damping ratio

CHECKPOINT: Problem Set 0, Q2–Q5 done by hand; you can solve $\ddot{x} = \pm \omega^2 x$ from memory and say what each sign means physically.

PHASE 4 Numerical integration — write your own

2–4 evenings

State vectors, Euler's method, RK4, then scipy's `solve_ivp`. You write Euler and RK4 yourself and watch Euler fail beautifully before you're allowed to use the library. §9.

LOOK UP: explicit Euler method · Runge–Kutta 4 · Dormand–Prince / RK45 · `scipy.integrate.solve_ivp` docs

CHECKPOINT: your RK4 matches the analytic small-angle pendulum to plotting accuracy; your Euler visibly inflates the energy of an undamped swing; your rig falls from 1° to 15° in ≈ 0.32 s in simulation, matching the doubling-time math.

PHASE 5 Simulate the full machine, open loop

1–2 evenings

Two coupled equations, three states (θ , $\dot{\theta}$, $\dot{\phi}$). Run it with $\tau = 0$, with constant τ , with a torque pulse. No controller yet — build trust in the plant first.

LOOK UP: nothing new — this phase is consolidation

CHECKPOINT: the momentum audit passes: $J \dot{\theta} + I_w \dot{\phi} - \int K_g \sin \theta \, dt$ stays constant through any torqued run (§6). Energy is flat when $\tau = 0$.

PHASE 6 **Control: your PID**

3–6 evenings

The spring lens becomes a design method: pole placement gives you starting gains from the model instead of knob-twiddling. Implement the discrete PID loop (sample, compute, hold), respect $\tau_{\max}$, then run the wheel-speed discovery experiment. §10.

LOOK UP: PID controller anatomy · integrator windup · derivative kick / derivative filtering · zero-order hold

CHECKPOINT: balances from a 5° release with realistic torque limits; recovers from pokes; you can state what each gain does in spring language; you have met the wheel-speed drift and explained it with the momentum lens.

PHASE 7 **Swing-up**

2–4 evenings, stretch

Energy-pumping control to bring the arm from hanging to nearly upright, then a hand-off ("catch") to your linear controller inside the catch cone. §11.

LOOK UP: energy-shaping control · solve_ivp event detection

CHECKPOINT: from hanging at rest, the sim swings up, catches, and balances — one continuous run.

PHASE 8 **Visualization & dashboard**

instructor's lane

Animated rig, live strip charts of θ , $\dot{\phi}$, τ , gain sliders, a poke button. I build this against whatever interface your simulator exposes.

CHECKPOINT: you demo the whole thing to your ME friend.

STRETCH **The sequels**

Friction identification · motor electrical model (back-EMF, speed-dependent torque ceiling) · sensor noise and filtering · full-state feedback / LQR and why it finds the wheel-speed fix automatically · someday, hardware.

Rotational mechanics kit

The foundation at office-hours pace: torque from scratch, inertia from one bolt on a stick, and the integral notation demystified. Everything in §3 is assembled from these and nothing else.

2.1 Torque — the turning currency

Your working definition is correct, and we keep it: **torque is rotational force** — the "how hard is this being twisted" number for a pivot. What we need next is how to *measure* it. Two everyday experiments pin it down:

- **The door.** The same push swings the door easily from the handle and barely moves it from two centimeters off the hinge. So a force's twisting effect grows with its distance from the pivot.
- **The seesaw — the quantitative one.** A ~ 300 N kid sitting 2 m out balances a ~ 600 N adult sitting 1 m out. Balance means the two twists cancel exactly: $300 \cdot 2 = 600 \cdot 1$. Half the force at double the distance is the *same* twist. That playground-verified trade selects the product form:

$$\tau = F \cdot r_{\text{push}}$$

lever law – r is where the force lands

Footnotes: only the force component perpendicular to the lever twists (general form $\tau = r F \sin \alpha$; the parallel component just tugs the pivot). Counter-clockwise positive, forever. Units: newton-meters.

Now the idea the rest of this section stands on. **Torque is a currency with a two-sided ledger:**

- **Supply** — every force applied to a body supplies torque about the pivot, priced by the lever law: F times *where the push lands*.
- **Demand** — a body being angularly accelerated demands torque. §2.2 computes that price, and it depends on *where the mass sits*.

The two sides meet in the law we are about to derive: supplied torque = (demand coefficient) $\times \theta$ — and the demand coefficient is what will get the name *moment of inertia*. Keep the sides straight and most torque confusion evaporates; §2.2 ends with the classic mix-up, resolved.

OPTIONAL DEPTH – WHY $F \cdot R$ IS NATURE'S BOOKKEEPING, NOT JUST A CONVENTION

Work = force $\times$ distance. Push tangentially with force F at radius r while the body turns by a small angle $d\theta$: your hand rides an arc of $r \cdot d\theta$ meters, so you did work $F \cdot (r \cdot d\theta) = (F \cdot r) \cdot d\theta$. In a straight line, work = force $\times$ distance; in rotation, work = torque $\times$ angle. $F \cdot r$ is the exchange rate between pushing in meters and twisting in radians. This also explains **grip-independence**: push hard close-in or gently far-out — the forces differ, but $F \cdot r$ (and the work per radian) is identical. That is why rotational bookkeeping is done in torque: forces depend on where you grab; torque doesn't.

2.2 One bolt on a stick: what a mass at radius r demands

The mission. We want rotation's version of $F = m \cdot a$ — a law shaped like

$$\tau = (\ ? \) \cdot \theta$$

the law we're hunting

so that whatever fills the "?" is rotation's version of mass. Here is the full translation dictionary between straight-line motion and rotation, with the one unknown:

STRAIGHT LINE	ROTATION
position x (m)	angle θ (rad)
velocity $\dot{x}$ (m/s)	angular velocity $\dot{\theta}$ (rad/s)
acceleration a (m/s ²)	angular acceleration $\ddot{\theta}$ (rad/s ²)
force F (N)	torque τ (N·m)
mass m (kg)	? — this subsection's question

The strategy — and why we bother computing the bolt's acceleration at all. At this point in the course we own exactly one law of motion: $F = m \cdot a$. It speaks in meters — real forces making real masses gain real m/s^2 — and it does not accept radians: θ is *not* the acceleration of any mass; it is a rate for an angle. So the only route to the rotational law is a detour through Newton's language: take the simplest spinning object — a massless stick, pivoted, one bolt of mass m glued at radius r — express the bolt's *actual* acceleration, let $F = m \cdot a$ price the force, then convert that force back into torque. Whatever multiplies θ at the end is the "?". We make this detour once; its output is what we use forever after.

- 1. Translate angle-language into meters.** The bolt rides a circle. Arc length is $s = r \cdot \theta$, so differentiating twice, its real acceleration along its path is $a = r \cdot \ddot{\theta}$. Note what this already says: two bolts sharing the same $\ddot{\theta}$ are *not* treated equally — the far one is dragged through more meters per second², because the same angular sweep covers a bigger circle. (Your observation exactly: rim points of a wheel out-accelerate hub points.)
- 2. Let Newton price the force.** To make mass m gain $a = r \cdot \ddot{\theta}$, something must push it with $F = m \cdot a = m \cdot r \cdot \ddot{\theta}$ — real newtons, delivered to the bolt through the stick's rigidity.
- 3. Convert that force back into torque.** The force lands on the bolt — at radius r from the pivot — so on the supply ledger it counts as $\tau = F \cdot r$ (lever law, §2.1).

$$\tau = (m r \ddot{\theta}) \cdot r = (m r^2) \ddot{\theta}$$

the "?" is $m \cdot r^2$

Watch it happen with numbers. Same bolt ($m = 1 \text{ kg}$), same angular acceleration ($\ddot{\theta} = 1 \text{ rad/s}^2$), two placements:

BOLT SITS AT	REAL ACCELERATION A = R $\ddot{\theta}$	FORCE NEEDED F = MA	TORQUE DEMANDED T = F $\cdot$ R
r = 1 m	1 m/s ²	1 N	1 N $\cdot$ m
r = 2 m	2 m/s ²	2 N	4 N $\cdot$ m

Double the radius $\rightarrow$ *quadruple* the demand, and the two doublings differ in kind: the first because the mass genuinely accelerates harder (step 1), the second because

the force it needs sits farther from the pivot, which corresponds to more torque (step 3). That is the honest origin of r^2 — and of your correct intuition that the same mass farther out needs more torque. Now quantified: not more by r ; more by r^2 .

PITFALL — THE TWO R'S: "FARTHER OUT IS EASIER" VS. "FARTHER OUT IS HARDER"

Both statements are true, with no contradiction, because their r 's live on opposite sides of the ledger:

- **Demand side — where the mass sits.** The same mass farther from the axis demands more torque for the same θ — as r^2 (this subsection).
- **Supply side — where the push lands.** The same force applied farther from the pivot supplies more torque — as r (the lever law). The door handle: fewer newtons for the same twist.

One r is the mass's address; the other is the push's address. A merry-go-round loaded with kids at the rim (heavy demand) is still easiest to push *from* the rim (best supply). In the bolt derivation the two r 's were the same number only because the force we tracked was the one delivered *to the bolt, at the bolt*.

2.3 What moment of inertia is — and isn't

Moment of inertia is **resistance to *changing* rotation — not to rotating**. Compare with ordinary mass: a heavy cart is not hard to *keep* moving (constant velocity is free — Newton's first law); it is hard to *speed up* and equally hard to *stop*. Same in rotation. "Is high I easier or harder to rotate?" is not a well-posed question until you say which you mean: start, keep, or stop. I answers *start* and *stop* — both get harder; *keep* is free (ideally). One object to anchor it: a playground merry-go-round — brutal to get going, brutal to stop, coasts forever. That is high I .

$$\tau = I\theta \quad \leftrightarrow \quad F = ma$$

the exact parallel

"Why would I have to push harder if my wheel's mass is at the rim?" Because of the r^2 of §2.2: a rim crumb costs $\sim r^2$ more torque per unit of wheel acceleration than a hub crumb. For our machine that is a *feature*, not a bug. The wheel's equation is $\ddot{\phi} = \tau/I_w$: big I_w means your motor spins the wheel up slowly — which sounds bad until you recall the wheel is a momentum bank (§6). For the same stored momentum $I_w\dot{\phi}$, the big- I_w wheel is turning *slowly*, far from the motor's rpm ceiling. Rim-heavy = bigger tank. Meanwhile the reaction shove on the arm equals τ regardless of I_w — the arm feels your torque, not your wheel's inertia.

2.4 Many crumbs: the sum, and what $\int r^2 dm$ really says

A rigid body is a bag of crumbs that all share the same θ and the same $\dot{\theta}$ — that is what "rigid" means. Each crumb i costs torque $m_i r_i^2 \cdot \ddot{\theta}$, and torque bills add:

$$\tau_{\text{total}} = (m_1 r_1^2 + m_2 r_2^2 + \dots) \ddot{\theta} \triangleq I \ddot{\theta} \quad \text{rigid body, fixed axis}$$

I is defined as that sum — mass $\times$ radius², crumb by crumb. (Watch the recap: I is a sum of $m \cdot r^2$ terms, *not* "torques times radii"; the torque statement is the separate line $\tau = I\ddot{\theta}$. That line, summed over the whole body, is Newton's second law in rotational form, and it generates half our model in §3.) When the mass is smeared out continuously, the sum is written

$$I = \int r^2 dm \quad \text{definition}$$

Read **dm** as "the mass of one tiny crumb." The whole expression is a recipe, not a mystery: chop the body into crumbs so small that each sits at a single radius; multiply each crumb's mass by its r^2 ; add them all up. It is §2.2 with infinitely many, infinitely small bolts. And in practice you *never* integrate "over mass" directly — you always convert dm into geometry first: pick a slicing coordinate, and write the crumb's mass in terms of *where it is* and *how thick it is*. Both worked examples below do exactly that, slowly.

2.5 How to read $\int_0^l$ — definite integrals in one lesson

The little numbers are **mileposts**: $\int_0^\ell (\dots) dr$ reads "sum the contributions as r sweeps from 0 (start) to ℓ (end)." That's all they are — where the tour of slices begins and where it stops. Such a start-to-end sum is called a *definite* integral, and the recipe for evaluating one has two steps:

- **Step A — find the antiderivative:** the function whose derivative is the thing you're summing. For r^2 it is $r^3/3$ — and you *verify* that the way you verify everything in this course, by differentiating the claim: $d/dr(r^3/3) = r^2$. ✓
- **Step B — plug in the end, plug in the start, subtract:**

$$\int_0^\ell r^2 dr = [r^3/3]_0^\ell = \ell^3/3 - 0^3/3 = \ell^3/3 \quad \text{evaluate}$$

Why subtract? The antiderivative works like an **odometer** — a running total of everything accumulated so far. The amount accumulated *between* two mileposts is the reading at the end minus the reading at the start. (Watch for a start reading that is negative, as in §2.8: subtracting a negative adds.)

2.6 Worked slowly: the rod about its end

Uniform rod, mass m , length ℓ , pivot at one end. Slice it like a baguette: the slice at distance r from the pivot has thickness dr . The rod is uniform, so its linear density is m/ℓ kilograms per meter, and the slice's mass is

$$dm = (m/\ell) \cdot dr \quad \text{crumb mass, via geometry}$$

Each slice contributes $r^2 \cdot dm$. Add every slice from pivot to tip, pulling the constant out front, then apply the §2.5 recipe and cancel:

$$I = \int_0^\ell r^2 (m/\ell) dr = (m/\ell) \cdot \ell^3/3 = m \ell^2/3 \quad \text{one } \ell \text{ cancels}$$

Sanity-check against cases needing no math: all the mass glued at the tip $\rightarrow m\ell^2$ (§2.2). All at the pivot $\rightarrow 0$ ($r = 0$; spinning costs nothing). All at the midpoint $\rightarrow m(\ell/2)^2 = m\ell^2/4$. Uniform rod: $m\ell^2/3$ — a bit *more* than the midpoint case, because r^2 over-weights the far half; the outer crumbs punch above their weight.

And if "the sum becomes an integral" still feels like a magic trick, make it concrete — approximate the rod as N equal chunks sitting at their centers and add them up (§2.4 style):

CHUNKS	SUM, IN UNITS OF $M\ell^2$
3	0.3241
10	0.3325
100	0.333325
∞ (the integral)	$1/3$ exactly

The integral is simply the place this staircase is climbing toward. One line of numpy in your notebook — yours to write — will reproduce the table.

2.7 Worked slowly: the disk about its center

PITFALL – TWO RADII, AND THEY MUST NOT SHARE A LETTER

R (capital, a fixed number) is the disk's outer radius — the whole disk, once and for all. r (lowercase, the sweeping variable) is where the particular thin ring under inspection sits; it runs from 0 to R as the tour proceeds. Writing the density with a lowercase r is the classic trap — density describes the whole disk, so it uses R .

For a disk spun about its center, the crumbs sharing a single radius form a **thin ring**. Two ingredients:

- **Areal density** (mass per unit area, a constant): $\sigma = \text{total mass} / \text{total area} = m/(\pi R^2)$.
- **The ring's area:** snip the ring at radius r , thickness dr , and unroll it into a straight ribbon: length = circumference = $2\pi r$, width = dr , so $dA = 2\pi r \cdot dr$. Note $2\pi r$ alone is a *length* — a circle of string has no area and no mass; the thin width dr is what makes it a sliver of area, exactly as the rod's slice needed its thickness.

Mass of the ring = density $\times$ area — and the π upstairs (circumference) kills the π downstairs (disk area), while the R^2 stays (it came from the density, a constant) and the lowercase r stays (the ring's own circumference talking):

$$dm = \sigma \cdot dA = [m/(\pi R^2)] \cdot 2\pi r dr = (2m/R^2) r dr \quad \pi \text{ cancels}$$

Now the same recipe as the rod — antiderivative of r^3 is $r^4/4$ (check: $d/dr(r^4/4) = r^3$ ✓), odometer at R minus odometer at 0 , then $R^4/R^2 = R^2$:

$$I = \int_0^R r^2 \cdot (2m/R^2) r dr = (2m/R^2) \cdot R^4/4 = \frac{1}{2} m R^2 \quad \text{solid disk}$$

Why only $\frac{1}{2}$? The inner mass is deadweight — small r . A **hoop** with all mass at the rim achieves the maximum, mR^2 . Real design guidance for future hardware: a reaction wheel wants to be a bicycle wheel, not a solid puck — rim-heavy maximizes I_w per gram.

✓ CHECKPOINT – THIRTY-SECOND SELF-TEST

Two wheels, same mass, same radius — one solid disk, one hoop. Same motor torque applied to each. Which spins up faster? Which makes the better reaction wheel, and why? Answer both crisply and you own this section.

ANSWERS

The disk spins up twice as fast (half the I , and $\ddot{\phi} = \tau/I$). The hoop is the better reaction wheel: double the I means double the momentum stored per rad/s of speed, so it reaches the motor's rpm ceiling with twice the momentum banked — a bigger tank (§2.3, §6).

2.8 Parallel-axis theorem — why exactly $+md^2$

$$I = I_{\text{com}} + m d^2 \quad \text{axis a distance } d \text{ from the COM}$$

The energy way. A body circling an off-center pivot does *two motions at once*: its center of mass rides a circle of radius d (speed $v = d\omega$), *and* the body spins about its own center at that same ω . Not convinced of the second part? The Moon orbits Earth always showing one face — to do that it must spin about its own axis exactly once per orbit. Orbiting rigidly *is* spinning. Kinetic energies add:

$$KE = \frac{1}{2} m(d\omega)^2 + \frac{1}{2} I_{\text{com}} \omega^2 = \frac{1}{2} (I_{\text{com}} + m d^2) \omega^2 \quad \text{two motions}$$

Whatever multiplies $\frac{1}{2}\omega^2$ is playing the role of inertia — so I about the pivot is $I_{\text{com}} + md^2$.

The algebra way (three lines). A crumb sits at distance ρ from the center of mass; the axis is offset by d , so the crumb's radius from the axis obeys $r^2 = (d + \rho)^2 = d^2 + 2d\rho + \rho^2$. Sum over crumbs, term by term: $\Sigma m \cdot d^2 = md^2$; the middle term is $2d \cdot \Sigma m\rho = 0$, because the center of mass is *defined* as the balance point where mass-weighted positions cancel; and $\Sigma m\rho^2 = I_{\text{com}}$. Total: $md^2 + I_{\text{com}}$. The cross term dying is the meaning of "center of mass." (Shown along the offset direction; the vector version runs identically.)

Cross-check, course-ethos style — the rod's end-inertia by a second, independent route. About its *center*, slicing from $-\ell/2$ to $+\ell/2$ (note the negative start milepost — subtracting a negative adds):

$$I_{\text{com}} = (m/\ell) [r^3/3]_{-\ell/2}^{+\ell/2} = (m/\ell)(\ell^3/24 + \ell^3/24) = m\ell^2/12 \quad \text{rod ab}$$

$$I_{\text{end}} = m\ell^2/12 + m(\ell/2)^2 = m\ell^2/12 + 3m\ell^2/12 = m\ell^2/3 \quad \checkmark \quad \text{matches §}$$

Same answer as the direct integral, by a completely different route. When two derivations agree, you're allowed to start believing. This theorem is how you'll assemble J for real geometry: each component's own inertia, plus md^2 for where it sits.

2.9 Newton's third law for torques

The motor's stator is bolted to the arm; its rotor is bolted to the wheel. When the stator torques the rotor by $+\tau$, the rotor torques the stator by $-\tau$. There is no way to torque the wheel without the arm feeling the recoil. This is the entire actuation principle of the machine — and of a cat rotating in free fall, a diver mid-somersault, and you swiveling an office chair by swinging your arms.

2.10 Angular momentum — the conserved currency

Angular momentum about a pivot is $H = I\omega$ for each spinning part (plus $m v r$ for parts orbiting the pivot). Its power comes from one law:

$$dH/dt = \Sigma \tau_{\text{external}} \quad \text{momentum law}$$

Only torques from *outside* the system count. Internal torques — like our motor acting between arm and wheel — come in third-law pairs and cancel in the sum. Choose your "system" cleverly and whole categories of unknown forces vanish from the bookkeeping. §3 uses exactly this trick; §6 builds a whole worldview on it.

2.11 Worked: the example rig's three numbers

One more idea before the arithmetic: the wheel appears in **two ledgers, and that is not double-counting**. Its *location* on the arm contributes $m_w L^2$ to J — it is cargo the arm must swing (§2.2's point mass, at $r = L$). Its *own spin* about its own axle is a separate motion with its own coordinate ϕ and its own inertia I_w (§2.7's disk). One object, two distinct rotations, two entries.

WORKED NUMBERS

- Arm (rod about its end): $I_a = m_a \ell^2 / 3 = 0.20 \cdot 0.15^2 / 3 = 1.50 \times 10^{-3} \text{ kg} \cdot \text{m}^2$
- Wheel carried at the tip (point mass for pivot purposes): $m_w L^2 = 0.30 \cdot 0.15^2 = 6.75 \times 10^{-3} \text{ kg} \cdot \text{m}^2$
- $J = 1.50 \times 10^{-3} + 6.75 \times 10^{-3} = 8.25 \times 10^{-3} \text{ kg} \cdot \text{m}^2$
- $I_w = \frac{1}{2} m_w r_w^2 = \frac{1}{2} \cdot 0.30 \cdot 0.05^2 = 3.75 \times 10^{-4} \text{ kg} \cdot \text{m}^2$
- Gravity stiffness (defined in §3): $K_g = (m_a \cdot 0.075 + m_w \cdot 0.15) \cdot 9.81 = 0.589 \text{ N} \cdot \text{m/rad}$

PITFALL – WHERE DOES THE WHEEL'S I_w LIVE?

Notice J contains the wheel's mass (as $m_w L^2$) but *not* its spin inertia I_w . That's because we track the wheel's spin with its own coordinate ϕ measured in the world frame. The wheel's kinetic energy splits cleanly: $\frac{1}{2} m_w (L\dot{\theta})^2$ for its center riding on the arm, plus $\frac{1}{2} I_w \dot{\phi}^2$ for its spin. If you instead measure wheel angle relative to the arm — which is what a motor encoder physically reports — cross-terms appear and I_w shows up in the arm equation too. Same physics, messier algebra. Problem Set 1, Q3 makes you do it both ways and prove they agree.

The model, by forces & torques

Newton–Euler derivation: one free-body diagram, one clever choice of system, two equations.

3.1 Conventions (decide once, engrave in stone)

- θ — arm angle from **upright**, counter-clockwise positive. $\theta = 0$ is the goal. The hanging rest position is $\theta = \pi$.
- ϕ — wheel rotation angle in the **world frame**, counter-clockwise positive.
- τ — motor torque applied **to the wheel**. The arm automatically receives $-\tau$.
- Dots are time derivatives: $\dot{\theta}$ is angular velocity, $\ddot{\theta}$ is angular acceleration.

3.2 The wheel's equation — a one-line free-body diagram

Isolate the wheel. Three things touch it: gravity (acting at its center), the axle bearing force (acting through its center), and the motor torque τ . Take torques about the wheel's own center: forces through the center have zero lever arm, so they contribute nothing. Only τ survives:

$$I_w \ddot{\phi} = \tau \tag{1}$$

Read it twice — it says something surprising. In the ideal model, the wheel's spin doesn't care about θ at all. The wheel is a pure torque-to-spin converter, and (as §6 will reframe it) a momentum bank with one deposit slot.

3.3 The arm's equation — pick the system so the junk cancels

The honest free-body diagram of the arm alone is a mess: the axle transmits the wheel's weight *and* the force needed to accelerate the wheel's center as it rides the arm. You'd have to solve for those internal forces just to eliminate them.

The professional move: take the **whole machine — arm plus wheel — as one system** and use the momentum law about the pivot O. Total angular momentum about O is the arm's spin, plus the wheel's center orbiting the pivot, plus the wheel's own spin:

$$H_O = I_a \dot{\theta} + m_w L^2 \dot{\theta} + I_w \dot{\phi} = J \dot{\theta} + I_w \dot{\phi} \quad (2)$$

Now count *external* torques about O. The pivot's reaction force acts at O — zero lever arm. The motor torque is internal — its third-law pair cancels. Gravity is the only survivor: the arm's weight acts at $\ell/2$ with lever arm $(\ell/2)\sin \theta$, the wheel's weight at L with lever arm $L \sin \theta$. Define the **gravity stiffness**

$$K_g \triangleq (m_a \ell/2 + m_w L) g \quad \text{N}\cdot\text{m}/\text{rad}$$

so the total gravity torque is $K_g \sin \theta$, pushing *away* from upright. The momentum law $dH_O/dt = \Sigma \tau_{\text{ext}}$ gives $J \ddot{\theta} + I_w \ddot{\phi} = K_g \sin \theta$; substitute (1) and the wheel term becomes τ :

THE MODEL — BOX IT LIKE A FINAL ANSWER

$$J \ddot{\theta} = K_g \sin \theta - \tau \quad (3)$$

$$I_w \ddot{\phi} = \tau \quad (1)$$

3.4 Interrogate it before you trust it

- **$\tau = 0$:** the arm is an ordinary free pendulum; the wheel coasts at constant speed. Sensible.
- **Balanced and still** ($\theta = 0, \dot{\theta} = 0, \tau = 0$): nothing moves. It's an equilibrium — an unstable one, as §5 will quantify.
- **Signs:** lean counter-clockwise ($\theta > 0$) and gravity's $+K_g \sin \theta$ pushes you further counter-clockwise — instability, correctly signed. Command $\tau > 0$ and the wheel accelerates counter-clockwise (1) while the arm receives $-\tau$, a clockwise shove (3). *To push yourself clockwise, hurl the wheel counter-clockwise.*

- **One knob, two effects:** τ appears in both equations with opposite roles. You cannot influence the arm without also spinning the wheel. That's underactuation, and it will matter (§6).

FIELD NOTE – FEEL IT IN A CHAIR

Sit on a swivel chair, feet off the floor, holding something moderately heavy. Swing it hard to the left; you rotate right. You are the arm; your arms are the motor; the weight is the wheel. Every sign in equation (3) is verifiable from a desk chair.

REALITY BOLT-ONS (ADD IN PHASE 6+, NOT BEFORE)

- **Pivot friction:** subtract $b_a\dot{\theta}$ from the right side of (3).
- **Wheel bearing friction:** acts on the *relative* speed ($\dot{\phi} - \dot{\theta}$): subtract $b_w(\dot{\phi} - \dot{\theta})$ in (1), add it in (3) — third-law pairs again.
- **Motor truth:** torque isn't free. A real motor's available torque falls as the wheel speeds up (back-EMF), roughly $\tau_{\text{avail}} = \tau_{\text{max}}(1 - |\dot{\phi}|/\omega_{\text{max}})$. Keep this in your back pocket for the §10 discovery experiment.

The model, by energy (Lagrange)

The same two equations from one scalar function and zero cleverness. This is the method that scales.

The Newton derivation needed a good idea (choose the system so the axle forces cancel). Lagrangian mechanics needs no ideas at all — that's its superpower. The recipe:

1. Choose coordinates. Ours: θ and φ .
2. Write kinetic energy T and potential energy V . Form the Lagrangian $\mathcal{L} = T - V$.
3. For each coordinate q , crank the handle:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{q}} \right) - \frac{\partial \mathcal{L}}{\partial q} = Q_q \quad (4)$$

where Q_q collects non-conservative inputs (motor torque, friction) acting on that coordinate.

4.1 The energies

$$T = \frac{1}{2} J \dot{\theta}^2 + \frac{1}{2} I_w \dot{\varphi}^2 \quad \text{kinetic}$$

$$V = K_g \cos \theta \quad \text{potential}$$

(V is the combined weight-times-height of arm and wheel, measured from the pivot; the same K_g from §3 appears because potential energy's slope *is* gravity torque: $-dV/d\theta = K_g \sin \theta$. Height is maximal at $\theta = 0$: the arm sits on top of an energy hill.)

For the motor: it applies $+\tau$ to the wheel and $-\tau$ to the arm, so $Q_\theta = -\tau$ and $Q_\varphi = +\tau$.

4.2 Crank the handle

θ coordinate: $\partial\mathcal{L}/\partial\theta = J\theta$, whose time derivative is $J\dot{\theta}$. And $\partial\mathcal{L}/\partial\dot{\theta} = -\partial V/\partial\dot{\theta} = K_g \sin \theta$. So (4) reads $J\ddot{\theta} - K_g \sin \theta = -\tau$ — equation (3). ✓

ϕ coordinate: $\partial\mathcal{L}/\partial\dot{\phi} = I_w\dot{\phi}$, time derivative $I_w\ddot{\phi}$. And $\partial\mathcal{L}/\partial\phi = 0$ — the Lagrangian doesn't contain ϕ at all. So (4) reads $I_w\ddot{\phi} = \tau$ — equation (1). ✓

No bearing forces, no axle forces, no system-choosing. They never entered the bookkeeping.

WHY ENGINEERS LOVE THIS – TWO REASONS

- **It scales.** For a double pendulum or a walking robot, Newton free-body diagrams become a thicket of internal forces. The Lagrangian recipe stays exactly three steps.
- **Conservation laws fall out free.** ϕ never appears in $\mathcal{L}$ (only $\dot{\phi}$ does) — such a coordinate is called *cyclic*, and its momentum $\partial\mathcal{L}/\partial\dot{\phi} = I_w\dot{\phi}$ is automatically conserved whenever its input τ is zero. The math just told us: cut the motor and the wheel's spin momentum freezes. §6 turns this observation into a control-relevant worldview.

Lens: it's a spring

Your ME friend is right — and the reason he's right is the deepest idea in this course: near an equilibrium, every mechanical system is springs, masses, and dampers.

5.1 Linearize: the small-angle approximation

Near upright, θ is small and $\sin \theta \approx \theta$ (in radians — the error is under 1% out to 14°). Equation (3) becomes

$$J \ddot{\theta} = K_g \theta - \tau \tag{5}$$

Compare with a textbook torsional spring–mass, $I \ddot{\theta} = -k\theta$, which oscillates forever at frequency $\sqrt{k/I}$. The *hanging* pendulum (measure θ' from straight down and linearize) is exactly that:

$$J \ddot{\theta}' = -K_g \theta' \tag{6} \text{ hanging}$$

A hanging pendulum is a mass on a rotational spring whose stiffness K_g is made of gravity: displace it, and a torque proportional to displacement pulls it back.

Now look at (5). Same equation, one sign flipped. The inverted pendulum is a spring with **negative stiffness**: displace it, and a torque proportional to displacement pushes it *further away*. Everything you know about springs carries over — you just have to follow the minus sign, and §8 shows it leads from sine waves to runaway exponentials.

THE PARTY TRICK (AND A REAL LAB TECHNIQUE)

The hanging oscillation frequency and the inverted blow-up rate are the *same number*: $\omega_n = \lambda = \sqrt{(K_g/J)}$. For the example rig, $\sqrt{(0.589/0.00825)} = 8.45$ rad/s — hanging, it swings with a 0.74 s period; inverted, errors grow by e every 118 ms, doubling every $\ln 2/\lambda \approx \mathbf{82\ ms}$. Someday, on hardware, you'll measure the swing period of the hanging arm with a stopwatch and thereby know K_g/J without a single CAD number. Falling and swinging are the same physics with opposite signs.

5.2 The controller is a software spring

Suppose your controller measures θ and $\dot{\theta}$ and commands

$$\tau = k_p \theta + k_d \dot{\theta} \quad \text{PD law}$$

Substitute into (5) and collect terms:

$$J \ddot{\theta} + k_d \dot{\theta} + (k_p - K_g) \theta = 0 \quad (7)$$

Stare at that. It is a mass–spring–damper: inertia J , damper k_d , net spring $(k_p - K_g)$. Your P gain is a **spring you wrote in software**, fighting gravity's anti-spring; your D gain is a **damper you wrote in software**. The machine becomes stable the moment the net stiffness turns positive:

YOUR FIRST TUNING RULE – FROM ALGEBRA, NOT KNOB-TWIDDLING

$$k_p > K_g \quad \text{and} \quad k_d > 0 \quad (8)$$

For the example rig: your proportional gain must beat 0.589 N·m/rad before balancing is even possible. Below that, no amount of derivative gain saves you — you've built a weaker spring than the hill you're standing on.

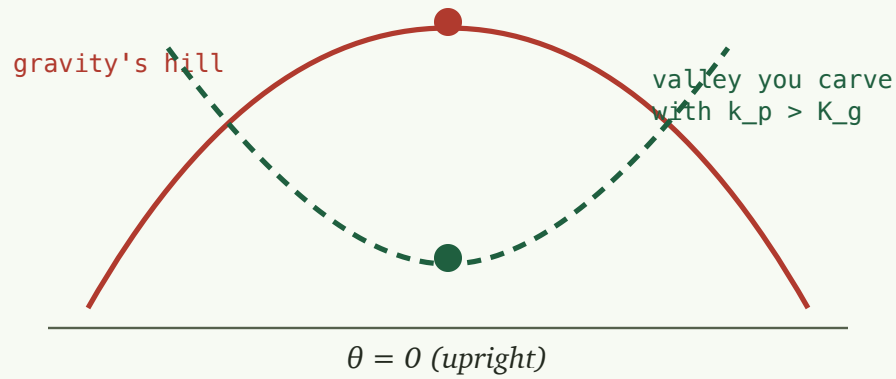


Fig. 2 – Uncontrolled, upright is the top of a potential-energy hill (solid): the ball rolls off. Feedback with $k_p > K_g$ reshapes the effective potential into a valley (dashed): the ball rolls home. Damping k_d decides how the rolling settles.

5.3 Design, not guesswork: from (ζ, ω) to gains

Equation (7) is the standard damped oscillator of §8.6, with closed-loop natural frequency and damping ratio

$$\omega_{cl} = \sqrt{(k_p - K_g)/J} \quad , \quad \zeta = \frac{k_d}{2\sqrt{J(k_p - K_g)}} \quad (9)$$

Run it backwards and you get a **design procedure**: choose the behavior, compute the gains.

$$k_p = K_g + J \omega_{cl}^2 \quad , \quad k_d = 2 \zeta \omega_{cl} J \quad (10)$$

WORKED NUMBERS – FIRST-CUT GAINS FOR THE EXAMPLE RIG

Choose $\zeta = 0.7$ (crisp but not ringing) and $\omega_{cl} = 2\lambda \approx 16.9$ rad/s (respond twice as fast as you fall):

- $k_p = 0.589 + 0.00825 \cdot 16.9^2 \approx \mathbf{2.9 \text{ N}\cdot\text{m/rad}}$
- $k_d = 2 \cdot 0.7 \cdot 16.9 \cdot 0.00825 \approx \mathbf{0.20 \text{ N}\cdot\text{m}\cdot\text{s/rad}}$

Now the honesty check: at a 5° lean (0.087 rad), the position term alone demands $\tau \approx 2.9 \cdot 0.087 \approx 0.26 \text{ N}\cdot\text{m}$ — but $\tau_{\max}$ is 0.10. Saturated. Stiffer gains shrink the region where your motor can actually deliver the spring you promised. This tension — aggression vs. authority — is the first real design tradeoff of Phase 6.

5.4 The deeper point your friend was making

Take *any* mechanical system near *any* equilibrium and Taylor-expand its energies: potential energy becomes a quadratic bowl (or hill) — a spring; kinetic energy is already quadratic — a mass. So near equilibrium, everything is masses, springs, and dampers; the only question is the sign and size of each. That is why mechanical engineers reflexively reach for the spring picture: it is not an analogy, it is the local truth of every machine. "Unstable" is just the word for a spring constant that came out negative — and feedback control is the business of selling the system a better spring.

Lens: it's a momentum bank

The conservation-law view. This lens predicts the one problem your PID cannot see — before you ever run the experiment.

From (2), total angular momentum about the pivot is $H_O = J\dot{\theta} + I_w\dot{\phi}$, and from the momentum law:

$$\frac{dH_O}{dt} = K_g \sin \theta \quad (11)$$

Read it as a bank statement. There are two accounts — the arm's ($J\dot{\theta}$) and the wheel's ($I_w\dot{\phi}$). The motor is the *transfer* button: it moves momentum between accounts and can never change the total. The only external party who can deposit or withdraw is **gravity, and only while the arm leans** ($\sin \theta \neq 0$). Balanced exactly upright, the total is frozen.

6.1 What the bank statement predicts

- **Balancing is momentum-shuffling.** Every gust that tips the arm lets gravity deposit momentum; your controller catches the arm by transferring that momentum into the wheel. The arm returns to vertical; the wheel keeps the change. *Each disturbance leaves a permanent residue of wheel speed.*
- **A PID on θ alone cannot see this.** Angle recovers perfectly, so the controller is satisfied — while $\dot{\phi}$ takes a random walk driven by disturbance history. In simulation the wheel just quietly spins faster. On hardware, back-EMF starves your torque as $\dot{\phi}$ grows, and one day the controller commands a shove the motor can no longer deliver. The failure is called **momentum saturation**, and it is why spacecraft carry thrusters or magnetorquers purely to "dump" wheel momentum.
- **The only drain is gravity.** To shed wheel momentum you must *lean the arm on purpose*, briefly, so gravity withdraws it. Sit with that: the fix for a too-fast wheel

involves deliberately tipping the pendulum. No angle-only controller would ever think of it; the momentum lens makes it obvious.

6.2 Your standing audit

✓ CHECKPOINT – THE MOMENTUM AUDIT (RUN IT FOREVER)

In every simulation you ever run of this machine, compute $J\dot{\theta}(t) + I_w\dot{\phi}(t) - \int_0^t K_g \sin \theta \, dt$. It must be constant to solver tolerance — through torque pulses, controller action, anything. If it drifts, your dynamics function has a bug or your solver tolerance is too loose. This one line has caught more sim bugs than any debugger.

This lens also seeds swing-up (§11): starting from hanging, the strategy is to rock the arm so that gravity's deposits arrive in phase — pumping energy and momentum in — until the arm can coast up to the catch zone.

Lens: it's a saddle point

The geometric view: dynamics as a landscape of flows, stability as the shape of the flow near a fixed point.

Stack the variables you'd need to freeze-frame the machine into a **state vector** $\mathbf{x} = (\theta, \dot{\theta}, \dot{\phi})$. (Wheel angle ϕ itself affects nothing — a consequence of it being cyclic — so three numbers suffice.) The model becomes one vector equation, $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \tau)$: from any point in state space, the dynamics tell you which way the point flows. A simulation is just following the arrows; §9 makes that literal.

Linearized about upright ($\sin \theta \rightarrow \theta$), the flow near the fixed point has three characteristic directions with growth rates — **eigenvalues** — that you will learn to read like an X-ray:

$$\{ +\lambda, \quad -\lambda, \quad 0 \} \quad \text{with} \quad \lambda = \sqrt{(K_g/J)} \approx 8.45 \text{ s}^{-1} \quad (12)$$

- **$+\lambda$ — the enemy.** A direction along which errors grow as $e^{+\lambda t}$. Its existence is what "unstable" means, and its size sets your reaction-time budget (82 ms to double).
- **$-\lambda$ — the free helper.** A direction along which errors die on their own. In the $(\theta, \dot{\theta})$ plane the two directions cross at the origin, making the flow a **saddle**: almost every trajectory sweeps in along the stable direction and gets flung out along the unstable one. The single knife-edge path that arrives at upright with exactly zero velocity is the *separatrix* — swing-up's landing strip.
- **0 — the sleeper.** A direction that neither grows nor decays: the wheel speed. Nothing in the uncontrolled physics regulates $\dot{\phi}$ — and (here is the quiet warning) feedback of θ and $\dot{\theta}$ alone leaves this eigenvalue exactly where it is. The momentum lens (§6) and the eigenvalue lens are telling you the same thing in two languages: *an angle-only controller stabilizes the saddle but leaves the wheel-speed direction adrift.*

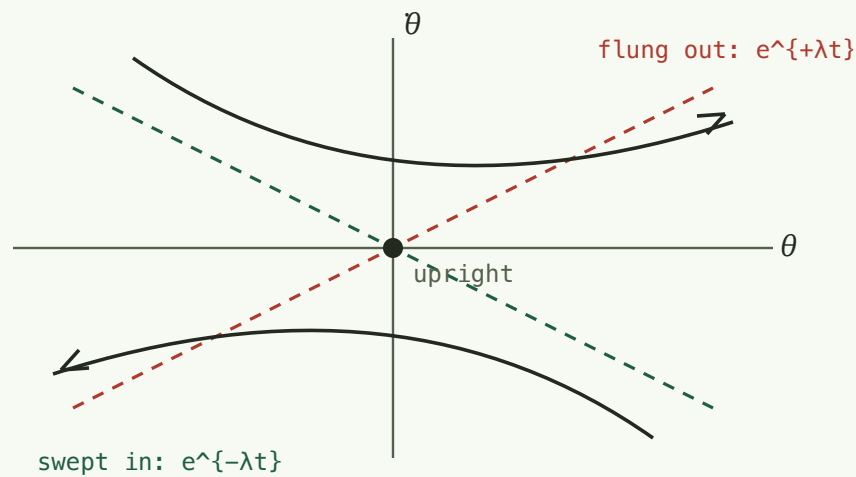


Fig. 3 – The saddle at upright in the $(\theta, \dot{\theta})$ plane. Trajectories approach along the stable (green) direction and escape along the unstable (red) one. Your controller's job, in this language: bend every arrow so it spirals into the origin. The single green in-bound path is the separatrix – arrive on it and the machine coasts to upright by itself.

Two facts to carry forward. First, **stabilization is eigenvalue surgery**: the PD law of §5 moves the $\pm\lambda$ pair into stable positions — that's what equation (7) accomplished. Second, the system is *controllable*: a feedback using all three states can place all three eigenvalues, sleeper included. You'll meet that as the fix at the end of Phase 6, and later as LQR. When you get to frequency response and transfer functions inside your PID reading, that's a fourth language for this same object; learn it as vocabulary, but the state picture is the native one here.

Differential equations from zero

Assuming you've never seen one. By the end you will solve every linear equation this machine can produce — by hand.

8.1 What a differential equation is

An ordinary equation constrains a *number*: $x^2 = 4$ pins x to ± 2 . A **differential equation** constrains a *function* — an entire curve through time — by relating the function to its own rates of change. When we write

$$\dot{x} = -2x$$

example

we are saying: find me a function $x(t)$ whose slope at every instant equals -2 times its current value. The unknown isn't a number; it's the whole trajectory. Newton's laws are differential equations by nature — force determines acceleration, acceleration is the second derivative of position, so physics hands you θ and asks you to recover $\theta(t)$. Equations (1) and (3) are precisely this: two coupled ODEs ("ordinary" = one independent variable, time).

YOUR DAY-ONE SUPERPOWER: VERIFICATION

You can *check* a claimed solution long before you can find one: differentiate it and substitute. Claim: $x(t) = 7e^{-2t}$ solves $\dot{x} = -2x$. Check: $\dot{x} = -14e^{-2t}$, and $-2x = -14e^{-2t}$. Equal — verified, no theory required. Every solution in this section can be checked this way in one line, and you should check each one. (This habit is also exactly how you'll validate simulations: propose, then verify.)

8.2 The atom: $\dot{x} = ax$

The most important ODE in the world: rate proportional to amount. Money compounding, populations growing, capacitors discharging, pendulums falling.

Solve it once, own it forever. Separate the variables — put every x on the left, every t on the right:

$$\frac{dx}{x} = a dt \rightarrow \int \frac{dx}{x} = \int a dt \rightarrow \ln x = a t + C$$

$$x(t) = x_0 e^{a t} \tag{13}$$

The constant of integration became the initial condition $x_0 = x(0)$ — every first-order ODE needs one launch datum. Behavior hinges entirely on the sign of a : negative a decays toward zero with time constant $1/|a|$ (63% of the gap closed per time constant); positive a grows without bound, doubling every $\ln 2/a$. Exponentials are the atoms of dynamics; everything linear that follows is built from them.

8.3 Second order, and the trick that turns calculus into algebra

Mechanics gives accelerations, so our equations are second order — and need *two* launch data (initial position and initial velocity; a thrown ball needs both). Take the frictionless spring–mass:

$$m \ddot{x} = -k x \tag{14}$$

The trick: since differentiating an exponential returns a multiple of itself, exponentials convert differential equations into polynomial equations. **Guess** $x = e^{\lambda t}$. Then $\ddot{x} = \lambda^2 e^{\lambda t}$, and (14) becomes $m \lambda^2 e^{\lambda t} = -k e^{\lambda t}$. The exponential never vanishes, so cancel it:

$$m \lambda^2 + k = 0 \Rightarrow \lambda = \pm i \sqrt{k/m} \quad \text{characteristic equation}$$

This polynomial is the **characteristic equation**; its roots λ are the eigenvalues of §7 — same objects, now unmasked. Here the roots are imaginary, and Euler's formula

$e^{i\omega t} = \cos \omega t + i \sin \omega t$ decodes them: imaginary exponents *are* rotation, i.e., oscillation. The real-valued solution is

$$x(t) = A \cos \omega_n t + B \sin \omega_n t, \quad \omega_n = \sqrt{k/m} \quad (15)$$

with A, B set by the two initial conditions. (Suspicious of i? Verify $x = \cos \omega_n t$ directly: $\ddot{x} = -\omega_n^2 \cos \omega_n t = -(k/m)x$. ✓) The hanging pendulum (6) is exactly this with $\omega_n = \sqrt{K_g/J}$: the 0.74-second swing.

8.4 Flip one sign: the mathematics of falling over

Now the inverted pendulum, equation (5) with the motor off: $J\ddot{\theta} = +K_g\theta$. Same guess, new characteristic equation:

$$J\lambda^2 = K_g \implies \lambda = \pm\sqrt{K_g/J} \quad \text{— real this time} \quad (16)$$

$$\theta(t) = A e^{+\lambda t} + B e^{-\lambda t} \quad (17)$$

Matching initial angle θ_0 and velocity $\dot{\theta}_0$ gives $A = \frac{1}{2}(\theta_0 + \dot{\theta}_0/\lambda)$ and $B = \frac{1}{2}(\theta_0 - \dot{\theta}_0/\lambda)$. Unless your initial condition is tuned to make A *exactly* zero, the $e^{+\lambda t}$ term eventually owns the motion. That is falling, as algebra. Two readings worth savoring:

- **The doubling clock.** Once the growing term dominates, errors double every $\ln 2/\lambda \approx 82$ ms. Whatever controller you write must observe, decide, and act meaningfully faster than that. Your control-loop rate requirement is hiding inside a square root of two physical constants.
- **The knife edge.** $A = 0$ means $\dot{\theta}_0 = -\lambda\theta_0$: approach upright with velocity *exactly* proportional-and-opposite to your lean and you coast asymptotically to balance. That solution is the separatrix of Fig. 3 — the landing strip swing-up aims for — and its measure-zero fragility is why "just aim carefully" is not a control strategy.

8.5 Add a damper: the equation your closed loop will be

$$m \ddot{x} + c \dot{x} + k x = 0 \rightarrow m\lambda^2 + c\lambda + k = 0 \quad (18)$$

The quadratic formula now runs the show. Real negative roots: two decaying exponentials (overdamped — molasses). A conjugate pair with negative real part: decaying oscillation (underdamped — ringing that fades). The boundary (critical damping) is the fastest no-overshoot return. Engineers normalize the whole family into

$$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x = 0 \quad (19)$$

Two dials describe every second-order system in the universe: ω_n , the speed of the dynamics, and ζ ("zeta"), the damping ratio — $\zeta < 1$ rings, $\zeta \geq 1$ doesn't, $\zeta \approx 0.7$ is the engineer's favorite compromise (fast, $\sim 5\%$ overshoot). Your PD-controlled pendulum, equation (7), **is** this equation with $m \rightarrow J$, $c \rightarrow k_d$, $k \rightarrow (k_p - K_g)$. Tuning gains and choosing (ζ, ω_n) are the same act — that's the pole-placement design of §5.3.

8.6 Push on it: forcing and offsets

Hang a constant disturbance torque d on the closed loop (a breeze, a slightly off-center mass): $J\ddot{\theta} + k_d\dot{\theta} + (k_p - K_g)\theta = d$. After transients die, $\ddot{\theta} = \dot{\theta} = 0$, so the system settles at $\theta_{ss} = d/(k_p - K_g)$ — a standing lean. Proportional control fights constant disturbances to a draw, never to zero. The classical cure is integral action: a term that accumulates error over time and keeps pushing until the error is truly gone. That's the "I" of your PID — and on this particular machine it has a twist you'll meet in the Phase 6 experiments (§10.5): a reaction wheel cannot hold a lean forever, because holding a lean means gravity deposits momentum into the wheel without limit.

8.7 The wall: why we go numerical

Everything above worked because the equations were *linear* — x and its derivatives appear plainly, never inside $\sin(\cdot)$ or multiplied together. Real equation (3) has $\sin \theta$. For large swings there is no elementary formula for $\theta(t)$ (the pendulum's period

genuinely depends on amplitude; the exact answer needs elliptic integrals — a name to recognize, not to study). Add a controller that switches, clips at $\tau_{\max}$, and updates every 5 ms, and closed-form solutions are gone for good. This is not defeat: it's the standard condition of engineering. The linear theory tells you *what to expect and how to design*; the computer tells you *what actually happens*. §9 builds that computer honestly.

Numerical integration & what "ode45" actually means

You will write Euler and RK4 yourself, watch Euler fail, and then use scipy's RK45 with justified confidence.

9.1 The universal interface: $\dot{x} = f(x)$

Every solver eats the same dish: a state vector and a function returning its derivative. Convert order-2 equations by promoting velocities to states. For the full machine, $x = (\theta, \dot{\theta}, \dot{\phi})$ and, reading straight off (3) and (1):

$$d/dt (\theta, \dot{\theta}, \dot{\phi}) = (\dot{\theta}, [K_g \sin \theta - \tau]/J, \tau/I_w) \quad (20)$$

Writing this function (your `derivs(t, x)`) is the moment the physics becomes software. It is also where most bugs live — hence the audits of §6 and §9.5.

9.2 Euler's method: the obvious thing

If you know where you are and which way you're flowing, step:

$$x_{n+1} = x_n + h f(x_n) \quad (21)$$

Follow the tangent for a small time h , re-evaluate, repeat. Taylor's theorem says each step's error is $O(h^2)$; over a fixed horizon the errors compound to $O(h)$ — halve the step, halve the error. Slow, but it works... except:

THE CLASSIC FAILURE — ASSIGNED, BECAUSE YOU MUST SEE IT ONCE

Simulate the undamped hanging pendulum with Euler and plot energy vs. time: it *grows*, and the swing spirals outward — the method manufactures energy from nothing. Geometry explains: undamped oscillation moves on a circle in $(\theta, \theta/\omega)$ coordinates, and a tangent line at any point of a circle lies strictly *outside* the circle. Every Euler step exits the orbit slightly. Any conclusion about damping or stability drawn from an Euler sim is contaminated by this artifact — which is why solver quality is not optional pedantry for this project.

9.3 Runge–Kutta 4: scouts before the march

Euler's sin is trusting the slope at the start of the step. RK4 sends scouts: evaluate the slope at the start, twice at trial midpoints, and once at a trial endpoint, then blend 1:2:2:1. Four evaluations per step buys global error $O(h^4)$ — halve h , gain $16\times$. This is the workhorse you'll hand-write in Phase 4; it will carry the whole project if you never touch a library.

9.4 ode45 / RK45, demystified

The name that's followed you here — MATLAB's `ode45`, scipy's `RK45` (both the Dormand–Prince pair) — adds one idea to RK4: **the step size should earn its keep**. From the same six slope evaluations, compute *two* answers of different accuracy (order 4 and order 5). Their disagreement estimates the local error, for free. Then:

$\text{err} > \text{tol}$: reject, shrink h and retry; $\text{err} \ll \text{tol}$: accept and grow h

The solver tiptoes through fast dynamics (a catch, a saturation event) and strides across calm coasting — accuracy where needed, speed where possible. That is the entire mystique of `ode45`: RK4's smarter sibling with an error gauge and a throttle. In scipy you'll live on `solve_ivp(f, t_span, x0, method="RK45", rtol=..., atol=...)`, plus two features made for this project: `dense_output` (smooth

trajectories for plotting/animation) and `events` (call me when $|\theta|$ crosses the catch cone — the clean way to switch from swing-up to balance).

9.5 The validation ritual

✓ CHECKPOINT – EVERY CLAIM ABOUT YOUR SIMULATOR, BACKED BY A TEST

1. **Equilibria hold:** start exactly upright (and exactly hanging), $\tau = 0 \rightarrow$ nothing moves.
2. **Analytic match:** hanging, 2° release $\rightarrow$ sinusoid at ω_n , period 0.744 s, overlaid on the formula from (15).
3. **Energy audit:** $\tau = 0 \rightarrow E = \frac{1}{2}J\dot{\theta}^2 + K_g \cos \theta$ flat; tighten `rtol` and watch residual drift shrink.
4. **Doubling clock:** release at 1° , $\tau = 0 \rightarrow$ reaches 15° in $\approx \ln(15)/\lambda \approx 0.32$ s, matching (17).
5. **Momentum audit** (§6.2) through any torqued run.
6. **Convergence:** tighten tolerances $10\times \rightarrow$ the trajectory stops changing to plotting accuracy.

9.6 Simulating a controller honestly

A real controller is not a formula living inside the physics — it's firmware that *samples*. Structure your closed-loop sim the way the eventual hardware works: every T_s seconds, read the state (later: add noise here), compute one PID output, clamp to $\pm\tau_{\max}$, then **hold τ constant** (zero-order hold) while the plant integrates forward one sample period. Loop. For our $\lambda \approx 8.5 \text{ s}^{-1}$ and closed-loop speeds around 17 rad/s, $T_s = 5\text{--}10 \text{ ms}$ (100–200 Hz) is comfortable; part of Phase 6 is discovering how slow you can sample before stability frays. Built this way, your controller code ports to a microcontroller nearly unchanged.

The control chapter

Design guidance and the experiments — the implementation and the discoveries are yours.

10.1 PID anatomy, in spring language

For this machine (setpoint $\theta = 0$), the natural sign convention is $\tau = k_p\theta + k_d\dot{\theta} + k_i\int\theta dt$, since positive τ pushes θ down via (3):

- **P — the spring.** Torque proportional to lean. Must out-muscle gravity's anti-spring: $k_p > K_g$, per (8).
- **D — the damper.** Torque proportional to lean *rate*; turns marginal oscillation into settling. In discrete time it amplifies sensor noise (a fast wiggle has a huge derivative), so real D-terms are filtered — and this is where most practical PID pain lives.
- **I — the grudge.** Accumulated error; erases standing offsets (§8.6). Comes with its own disease, *windup*: while τ is saturated the integral keeps growing on error it cannot act on, then overshoots violently when authority returns. Cure: clamp the integral (anti-windup) whenever output saturates.
- **Discrete form** (what you'll implement, at sample k , period T_s): integral $s_k = \text{clamp}(s_{k-1} + \theta_k T_s)$; derivative from differences, $d_k = (\theta_k - \theta_{k-1})/T_s$, passed through a low-pass filter; output clamped to $\pm\tau_{\max}$.

PITFALL — SIGN DISCIPLINE

If you write a generic library-style PID on error $e = \theta_{\text{ref}} - \theta$, its output needs a minus sign before becoming τ (compare with equation 3). Settle it with a static test before any dynamics: lean the sim $+5^\circ$, freeze time, print τ . It must be positive — wheel accelerating counter-clockwise, arm shoved clockwise, toward upright. Thirty seconds of test; hours of not chasing a "mysterious instability."

10.2 A tuning procedure that isn't folklore

1. **Start from the model, not from zero.** Pick $\zeta = 0.7$, $\omega_{cl} \approx 1.5-2\lambda$; compute k_p , k_d from (10). You already have numbers: ≈ 2.9 and 0.20 for the aggressive choice.
2. **Check authority.** Catch cone: gravity beats the motor beyond $\sin \theta = \tau_{\max}/K_g$ — about 9.8° for the example rig, *independent of gains*. Then check where your k_p alone saturates ($2.9 \cdot \theta > 0.10$ at just 2°). If the promised spring exceeds the motor almost everywhere, soften ω_{cl} or raise $\tau_{\max}$ and rerun. Design is this loop.
3. **Probe with steps and pokes.** Release from 3° ; poke with a 10 ms torque pulse. Watch overshoot and settling against the (ζ, ω_{cl}) you ordered — the sim should echo the theory within nonlinear-and-saturation fuzz. When it doesn't, that gap is a finding, not an annoyance.
4. **Sweep one gain at a time** and keep a lab-notebook table: k_p low $\rightarrow$ lazy or lost; k_d low $\rightarrow$ ringing; k_d high $\rightarrow$ noise amplification (add sensor noise and see it); k_i — see the experiment below.

10.3 The plot twist you now get to discover

EXPERIMENT – PREDICT, THEN RUN

With your PD balancing nicely, poke the arm three times, a second apart. Before running: use §6 to predict the wheel-speed trace $\dot{\phi}(t)$. Then add the motor-truth model (torque fading to zero as $|\dot{\phi}| \rightarrow \omega_{\max}$) and keep poking. Watch what fails, and when.

Then the challenge of the phase: **modify your controller so the wheel speed is also brought home**, using only quantities you can justify. You have two lenses that already explain the disease (§6, §7); let them prescribe. Try things. Simulate both signs of anything you add.

STUCK AFTER HONEST ATTEMPTS? ONE NUDGE

Gravity is the only drain. Shedding wheel momentum therefore requires the arm to lean, briefly, on the correct side. Consider letting the controller feel $\dot{\phi}$ — a third feedback term — and reason about which sign of that term causes a lean that drains the wheel. One sign works beautifully; the "obvious" one from force-intuition destabilizes. If you later learn LQR (Brunton's bootcamp), you'll find it hands you this term, correct sign and size, automatically — a satisfying reason to learn it.

10.4 What your PID cannot do (so you aren't surprised)

Angle-only PID stabilizes the saddle but leaves the sleeper eigenvalue at zero (§7): wheel speed integrates disturbance history. The integral term, meanwhile, fights standing disturbances by holding a lean — but on this machine a held lean pumps the wheel forever (§6). These aren't bugs in your code; they're truths about information: a controller can only regulate what it can see. When your experiments surface both, you've extracted everything PID has to teach on this machine — and you'll know exactly why full-state feedback exists.

Swing-up (preview)

Phase 7. Read now for orientation; work it after balance is solid.

From hanging, the arm needs energy $E^* = K_g$ (taking upright-at-rest as the target, measured by $E = \frac{1}{2}J\dot{\theta}^2 + K_g \cos \theta$). The arm subsystem's energy changes only through the reaction torque: $dE/dt = -\tau\dot{\theta}$. So a controller that chooses τ with sign opposite to $\dot{\theta} \cdot (E^* - E)$ pumps energy in whenever the arm is short of it — push in phase with the swing, like pumping a playground swing — and the wheel is your handle. Saturate at $\pm\tau_{\max}$, monitor E , and when the state enters the catch cone with modest $\dot{\theta}$ (a `solve_ivp` event), hand off to your balance controller. Expect the hand-off to be the fussy part; the momentum lens explains the wheel-speed state you arrive with.

Problem sets

PS0 gates the whole course. Answers fold open — after you've committed ink.

Problem Set 0 — entry ticket (Phases 1–3)

1. Compute I_a , I_w , J , and K_g for the example rig from the §0 table, by hand.
2. Compute λ , the doubling time, and the hanging period. State the coincidence connecting the last two.
3. Verify by substitution that $\theta(t) = \theta_0 \cos(\omega_n t)$ solves the hanging equation (6).
4. Verify that $\theta(t) = \theta_0 e^{\lambda t}$ solves the inverted equation (5) with $\tau = 0$. Starting at 1° , how long until 15° (where small-angle theory starts lying)?
5. Derive condition (8) from equation (7): why must k_p exceed K_g , and what goes wrong with $k_d = 0$ exactly?
6. Catch cone: with $\tau_{\max} = 0.10 \text{ N}\cdot\text{m}$, beyond what angle is balancing impossible for *any* controller? At what angle does the P-term alone ($k_p = 2.9$) hit the limit?
7. (*first notebook work — yours*) Hand-run Euler on $\dot{x} = -x$, $x(0) = 1$, $h = 0.1$, ten steps; compare against e^{-1} . Predict the error at $h = 0.05$, then verify.
8. (*notebook*) Euler-simulate the undamped hanging pendulum from 20° ; plot $E(t)$. Explain the drift with the tangent-exits-the-circle argument, then repeat with your RK4 and compare.

ANSWERS – PS0

1. $I_a = 1.50 \times 10^{-3}$, $I_w = 3.75 \times 10^{-4}$, $J = 8.25 \times 10^{-3} \text{ kg}\cdot\text{m}^2$; $K_g = 0.589 \text{ N}\cdot\text{m}/\text{rad}$.
2. $\lambda = 8.45 \text{ s}^{-1}$; $t_{\text{double}} = \ln 2/\lambda \approx 82 \text{ ms}$; $T = 2\pi/\omega_n \approx 0.744 \text{ s}$; $\omega_n = \lambda$ — the swing rate and the fall rate are the same $\sqrt{(K_g/J)}$.
3. $\theta = -\omega_n^2 \theta_0 \cos(\omega_n t) = -(K_g/J)\theta$. ✓
4. $\theta = \lambda^2 \theta_0 e^{\lambda t} = (K_g/J)\theta$. ✓ $t = \ln(15)/\lambda \approx 0.32 \text{ s}$.
5. Net stiffness ($k_p - K_g$) must be positive or the characteristic roots include a positive real one (still a saddle, just relabeled). With $k_d = 0$ the roots are purely imaginary: an undamped oscillation that never settles, and any real-world lag tips it unstable.
6. $\sin \theta = 0.10/0.589 \rightarrow \theta \approx 9.8^\circ$. P-term saturates at $\theta = 0.10/2.9 \approx 0.034 \text{ rad} \approx 2.0^\circ$.
7. Euler: $0.9^{10} \approx 0.3487$ vs $e^{-1} \approx 0.3679$ — about 5.2% low. Halving h roughly halves the error (first-order method): $0.95^{20} \approx 0.3585$, about 2.6% low.
8. Your plot; energy should climb steadily under Euler and hold flat (to tolerance) under RK4.

Problem Set 1 — mechanics (Phase 2)

1. Derive the rod-about-end result $m\ell^2/3$ from the integral definition, then re-derive it with the parallel-axis theorem from the rod-about-center value $m\ell^2/12$. (§2.6 and §2.8 do both — the exercise is reproducing them closed-book.)
2. Your wheel isn't a perfect disk (spokes, hub, motor rotor). Describe how you'd build I_w for real hardware from components + parallel axis, and why only the spinning parts count in I_w while *all* the tip mass counts in J .
3. Re-derive the model using the encoder's coordinate $\psi = \varphi - \theta$ (wheel angle relative to arm). Show the θ -equation becomes $(J + I_w)\ddot{\theta} + I_w\ddot{\psi} = K_g \sin \theta$ and that it contains the same physics as (3) and (1) combined.

4. Constant $\tau = 0.05 \text{ N}\cdot\text{m}$ for 0.5 s from rest near upright, arm free. Using the bank statement (11) and assuming θ stays within $\sim 2^\circ$, estimate the wheel speed and the arm's rotation rate at the end. (Hint: total H barely changes; the accounts must offset.)
5. Explain, chair-and-weight style, why a constant wheel *speed* (however huge) exerts no torque on the arm — only wheel *acceleration* does. Where does this show up in the equations?

SELECTED ANSWERS – PS1

3. With $\phi = \theta + \psi$: $T = \frac{1}{2}(J + I_w)\dot{\theta}^2 + I_w\dot{\theta}\dot{\psi} + \frac{1}{2}I_w\dot{\psi}^2$; the motor now works on ψ alone ($Q_\psi = \tau$, $Q_\theta = 0$). Cranking Lagrange: θ : $(J+I_w)\dot{\theta} + I_w\dot{\psi} - K_g \sin \theta = 0$; ψ : $I_w(\dot{\theta} + \dot{\psi}) = \tau$, i.e., $I_w\ddot{\phi} = \tau$ — equation (1) again; substitute back and the θ -equation collapses to (3). Same machine, either bookkeeping.
4. Wheel: $\dot{\phi} = \tau t / I_w = 0.05 \cdot 0.5 / 3.75 \times 10^{-4} \approx 67 \text{ rad/s}$ ($\approx 640 \text{ rpm}$). With $H \approx 0$ (gravity's deposits tiny near upright), $J\ddot{\theta} \approx -I_w\ddot{\phi} \rightarrow \dot{\theta} \approx -3.0 \text{ rad/s}$: the arm counter-rotates, hard. A constant torque is a lousy balancer but a great actuator-authority demo.
5. τ appears only alongside $\ddot{\phi}$. At constant $\dot{\phi}$, the motor torque is (ideally) zero and the reaction on the arm is zero: stored spin is potential authority, not force. This is exactly why momentum saturation is silent until you need the wheel to accelerate and it can't.

Problem Set 2 — ODEs (Phase 3)

1. Solve $\dot{x} = -2x$, $x(0) = 3$; sketch; mark the time constant.
2. Solve $\ddot{x} + 4\dot{x} + 29x = 0$ via the characteristic equation (roots $-2 \pm 5i$); write the real solution and identify ω_n , ζ , and the decay envelope.
3. For the example rig's first-cut gains ($k_p = 2.945$, $k_d = 0.195$), invert (9): recover ω_{cl} and ζ , confirming $\approx (16.9 \text{ rad/s}, 0.70)$.

4. For (17), find the initial conditions with $\theta_0 = 3^\circ$ that ride the knife edge, and compute where the arm is after 0.3 s if θ_0 misses that target by 1%.
5. Show that with forcing d , the settled solution of §8.6 is $\theta_{ss} = d/(k_p - K_g)$, and compute it for $d = 0.01 \text{ N}\cdot\text{m}$ with your gains.

Problem Set 3 — experiments (Phases 4–6, in your notebook)

1. Run the full §9.5 validation ritual; screenshot each audit into a "sim birth certificate" cell.
2. Reproduce the doubling clock numerically from 0.5° , 1° , 2° — show $t(15^\circ)$ shifts by $\ln 2/\lambda$ per halving of the start angle.
3. Close the loop at $T_s = 5 \text{ ms}$; verify the step response echoes your ordered (ζ, ω_{cl}) . Then stretch T_s until it breaks and report the failure mode.
4. The §10.3 poke experiment, with prediction written down first.
5. Add Gaussian angle noise (say $\sigma = 0.2^\circ$) and watch the D-term chew; add the filter; document the tradeoff you chose.

Reading list, glossary, notation

Annotated reading list

- **3Blue1Brown** — "Differential equations" series (YouTube). The geometric picture of ODEs and phase space. Watch alongside §8; the pendulum episode is literally our machine.
- **Åström & Murray** — *Feedback Systems*. The modern control intro; free PDF from the authors. Chapters 1–3 now, the PID chapter during Phase 6.
- **Brian Douglas** — **Control Systems Lectures (YouTube)**. The friendliest PID and classical-control intuition on the internet. Phase 6 companion.
- **Steve Brunton** — **Control Bootcamp (YouTube)**. State-space, eigenvalues, controllability, LQR. Watch after your PID works to see where the road continues.
- **Block, Åström & Spong** — *The Reaction Wheel Pendulum*. A ~100-page monograph about exactly this device. Read *after* deriving the model yourself, then enjoy translating their notation into yours (that translation is a real exercise, not a chore).
- **Tedrake** — *Underactuated Robotics* (underactuated.mit.edu). Free MIT course. Graduate-level dessert: energy shaping, acrobots, why underactuation is a career's worth of interesting.
- **MIT OCW 18.03 (Mattuck) or Tenenbaum & Pollard**. If §8 leaves you hungry for the full ODE course, these are the classic routes.
- **SciPy docs** — `solve_ivp`. Read the whole page once, especially `rtol/atol`, `dense_output`, `events`.
- **Watch: ETH Zürich "Cubli"; REM-RC reaction-wheel builds (YouTube)**. Motivation, and a preview of hardware reality.

Glossary

State / state vector

The minimal set of numbers that freeze-frames a system — here $(\theta, \dot{\theta}, \ddot{\theta})$. Knowing the state and the input determines the entire future.

Equilibrium

A state where nothing changes: upright (unstable), hanging (stable).

Linearization

Replacing $\sin \theta$ by θ near an operating point; trades global truth for local solvability. Valid to $\sim 1^\circ$ below 14° .

Eigenvalue / pole / characteristic root

Three names, one object: the growth rates λ in $e^{\lambda t}$. Negative real part = decay, positive = instability, imaginary part = oscillation.

Natural frequency ω_n ; damping ratio ζ

The two dials of any second-order system: how fast, and how ring-y ($\zeta < 1$ overshoots; $\zeta \approx 0.7$ is the engineer's default).

Time constant / doubling time

$1/|\lambda|$ for decay; $\ln 2/\lambda$ for growth. The rig's 82 ms doubling time is its difficulty rating.

Separatrix

The knife-edge trajectory that just barely reaches upright; boundary between "falls back" and "goes over." Swing-up's landing strip.

Underactuated

Fewer actuators than degrees of freedom (here 1 vs 2); you must recruit the natural dynamics to do the rest.

Zero-order hold (ZOH)

Holding the actuator command constant between samples — how digital controllers actually touch continuous plants.

Integrator windup / anti-windup

The I-term accumulating during saturation, then overshooting; cured by clamping the integral while output is limited.

Derivative kick / filtering

The D-term's noise amplification (and spike on setpoint changes); tamed with a low-pass filter or differentiating the measurement.

Momentum saturation

Wheel speed reaching the motor's limit so no further torque is available in that direction; the reaction-wheel disease (§6).

Dead-band (friction)

The cone $|\psi| < F_c/k_p$ where the controller's spring cannot break Coulomb friction, so the arm parks off-target (§14.1).

RK45 / ode45 / Dormand–Prince

An adaptive Runge–Kutta: paired 4th/5th-order estimates whose disagreement steers the step size.

Event detection

Solver feature that stops/flags integration when a condition crosses zero — e.g., entering the catch cone.

Catch cone

The angle range where available torque can still beat gravity: $\sin \theta \leq \tau_{\max}/K_g$.

Cyclic coordinate

A coordinate absent from the Lagrangian (our φ); its momentum is conserved when unforced.

LQR

Linear-quadratic regulator: full-state feedback with optimally chosen gains. The systematic version of what your final controller gropes toward.

Notation table

SYMBOL	MEANING	UNITS
$\theta, \dot{\theta}, \ddot{\theta}$	arm angle from upright (CCW+), its rate, its acceleration	rad, rad/s, rad/s ²
$\phi, \dot{\phi}, \ddot{\phi}$	wheel angle in the world frame, rate, acceleration	rad, rad/s, rad/s ²
$\tau, \tau_{\max}$	motor torque on the wheel (reaction $-\tau$ on arm); its limit	N·m
J	total inertia about the pivot (arm + wheel-as-point-mass)	kg·m ²
I_w	wheel spin inertia about its own axis	kg·m ²
K_g	gravity stiffness ($m_a \ell / 2 + m_w L$)g	N·m/rad
λ, ω_n	fall rate / swing frequency, both $\sqrt{(K_g/J)}$	s ⁻¹ , rad/s
ζ, ω_{cl}	closed-loop damping ratio and natural frequency (your design choices)	—, rad/s
k_p, k_i, k_d	your PID gains (software spring, grudge, damper)	N·m/rad, N·m/(rad·s), N·m·s/rad
H_O	total angular momentum about the pivot, $J\dot{\theta} + I_w\dot{\phi}$	kg·m ² /s
T_s	controller sample period	s
b_a, b_w	pivot and wheel-bearing viscous friction (Phase 6+ bolt-ons)	N·m·s/rad

Pathology reports from the sim lab

Real issues met while building the simulation — each diagnosed on paper with the lenses of §5–§7, each prediction checked against a measured run. Every number below uses Rohan's build, not the §0 example rig: 5 m, 5 kg arm; 8 kg hoop of radius 2 m at the tip $\rightarrow J = 241.7 \text{ kg}\cdot\text{m}^2$, $K_g = 514.5 \text{ N}\cdot\text{m}/\text{rad}$, $I_w = 32 \text{ kg}\cdot\text{m}^2$; Coulomb pivot friction $F_c = 1.0 \text{ N}\cdot\text{m}$; controller $\tau = k_p\psi + k_d\dot{\psi} + \tau_{\text{grav}}$ — PD plus *gravity feedforward* (τ_{grav} equals the current gravity torque, so its reaction cancels gravity on the arm), with ψ the lean from upright.

The method behind every result here is the same two-step move, and it is the single most transferable skill in this section. Step one: substitute the control law into the arm equation and simplify — the feedforward cancels gravity, leaving a plain mass–spring–damper with friction:

$$J\ddot{\psi} + k_d\dot{\psi} + k_p\psi = -F_c \text{sgn}(\dot{\psi}) \quad (23) \text{ closed loop, near upright}$$

Step two: interrogate that one line with the standard tools — the (ζ, ω) dials of §8.5 for the dynamics, a static torque balance for stalls, and the §6 bank statement for the wheel. No simulation was consulted to make any prediction below; the simulation only *graded* them.

14.1 Case: the arm parks off-vertical — the Coulomb dead-band

Symptom. With $k_p = 4$, $k_d = 25$, the arm climbs from 45° , then stalls **13.8° past upright** and stays there forever. Raising the gains to $k_p = 350$, $k_d = 250$ made it balance visually dead-vertical. Was that a fluke?

Diagnosis from (23). Ask the static question: the arm is at rest at lean ψ — can it move? Gravity is already cancelled by the feedforward, so the only torque trying to close the error is the software spring, $k_p\psi$. Coulomb friction resists with up to F_c regardless of how small the motion would be. Motion is impossible whenever the spring can't break friction:

$$|\psi| < F_c / k_p \triangleq \psi_{db} \quad (24) \text{ dead-band}$$

Inside that cone the controller is *shouting into a pillow*: it commands torque, the wheel spins up, the reaction lands on the arm — and static friction eats all of it. Where exactly the arm parks inside the cone depends on how it arrived (friction is hysteretic); anywhere inside is a legal resting place.

QUANTITY	$K_p = 4, K_d = 25$	$K_p = 350, K_d = 250$
dead-band $\psi_{db} = F_c/k_p$	0.25 rad = 14.3° (measured stall: 13.8°)	0.0029 rad = 0.16° ("perfectly vertical" — to the eye)
$\omega_{cl} = \sqrt{(k_p/J)}$	0.13 rad/s (49 s period)	1.20 rad/s (5.2 s period)
$\zeta = k_d/(2\sqrt{(k_p J)})$	0.40	0.43
settling $\approx 4/(\zeta\omega_{cl})$	~ 190 s	~ 8 s

(Note the feedforward variant of §5.3's formulas: because gravity is already cancelled, $\omega_{cl} = \sqrt{(k_p/J)}$ with no $-K_g$, and there is no $k_p > K_g$ threshold. The feedforward pre-paid gravity's bill.)

So is the stiff-gain cure real? Yes — with an asterisk. The offset was not *eliminated*; it shrank by exactly the predicted factor $4/350 \approx 1/87$, from 14.3° to 0.16° — below what the eye (or a 500-pixel GIF) can resolve. This is genuine, textbook servo engineering: precision machines fight stiction with stiffness first, and the residual offset scales as $1/k_p$. The asterisk is what stiffness costs on real hardware, none of which this sim yet charges for: **torque** ($k_p = 350$ commanded ~ 800 N·m early in this run — a real motor saturates, shrinking the region where the promised spring exists at all), **noise** ($k_d = 250$ multiplies every tremor of angle-measurement noise into torque chatter), and **sample rate** (fine here: $\omega_{cl}T_s \approx 0.012$). The classical alternative is integral action — it removes the offset *exactly*, but Coulomb friction plus an I-term is the canonical recipe for a slow limit cycle (the integrator winds until it breaks friction, the arm lurches through target, repeat — "hunting"). Stiffness, integral, and friction compensation are the three rungs; you climbed the first.

PITFALL – THE FRICTION MODEL ITSELF

Signum friction at a fixed time step chatters by construction (the sign flips every step near rest), a `v == 0` float check essentially never fires after the first step, and real stiction exceeds sliding friction (stick-slip). If Coulomb friction stays in the model, keep it knowingly; the numerically-kind, bearing-realistic bolt-on is viscous, $-b_a\dot{\theta}$ (§3.4), which has *no* dead-band — it fades to zero with speed instead of holding a grudge at rest.

14.2 Case: the wheel bill — why stiffer gains slowed the wheel 20×

Symptom. Same gain change, opposite-feeling effect: with $k_p = 4$ the wheel routinely hit ~ 200 rad/s (measured: peak 217, and at $t = 100$ s it read 97 rad/s *and was still ramping*). With $k_p = 350$ it idles near 10 rad/s. Why would a *stronger* controller use the wheel *less*?

Diagnosis from the bank statement (11). Integrate it over the whole run. The arm starts at rest and ends at rest, so its account $J\dot{\theta}$ contributes nothing net, and (up to a small friction impulse) everything gravity ever deposited is parked in the wheel:

$$I_w \dot{\phi}_{\text{end}} = \int (\text{gravity torque}) dt = K_g \int \sin \psi dt \quad (25) \text{ rest-to-rest}$$

The wheel's speed is the time-integral of the lean. Not of the gains, not of the torque you commanded — of how far, and for how long, the arm was off vertical. Gravity deposits momentum only while $\sin \psi \neq 0$, and the motor (being internal) can only decide *which account* holds it. Now read both runs through (25):

- **Soft gains charged twice.** First, the *transit*: a 49-second-period crawl spends tens of seconds at large lean — order-of-magnitude $\int$: an average $\sin \psi \approx 0.5$ held for ~ 30 s costs $514.5 \times 0.5 \times 30 \approx 7,700$ N·m·s, i.e. ≈ 240 rad/s of wheel — right at the measured 217 peak. Second, and worse, the *stall tax*: parked at 13.8° , gravity deposits $K_g \sin(13.8^\circ) \approx 123$ N·m *continuously*, and the feedforward dutifully shovels it into the wheel at $\ddot{\phi} = 123/32 \approx 3.8$ rad/s² —

forever. The " ~ 200 rad/s wheel" was not an equilibrium. It was a running meter.

- **Stiff gains starve the integral three ways.** The transit shrinks from ~ 30 s to ~ 8 s; the lean during it is smaller; and the overshoot wiggles about vertical alternate the *sign* of $\sin \psi$, so gravity claws back part of what it deposited (the ledger refunds). What remains is the shrunk stall tax: inside the new 0.16° dead-band the total torque reaching the wheel is at most $(K_g + k_p) \cdot \psi_{db} \approx 864.5 \times 0.0029 \approx 2.5$ N·m, a residual creep of ≤ 0.08 rad/s² — fifty times slower. A few rad/s from the quick transit plus a slow creep $\approx$ the observed ~ 10 rad/s. ✓

THE MORAL – PRECISION IS MOMENTUM HYGIENE

Gains never touch the wheel directly; they shape the *trajectory of the lean*, and equation (25) converts lean-time into wheel speed. Anything that makes the arm straighter, sooner, starves the wheel. Corollary: the creep never truly stops while any offset remains — only exact vertical (unreachable under Coulomb friction) or a controller that can *see* $\dot{\phi}$ (§10.3's third term; LQR) closes the meter. And on hardware, the soft-gain run's unbounded ramp ends one way: momentum saturation (§6), arriving exactly when the back-EMF model gets added.

14.3 Case: issues that looked like physics but were the camera

Three findings from the same debugging session, all in the visualization pipeline rather than the dynamics — recorded because each one masqueraded as a physics problem:

- **GIF clock skew.** GIF frame delays are stored in 10 ms units. Requesting 30 fps (33.3 ms) silently stored 30 ms/frame, so a 100 s run played back in 90 s — everything 11% fast, and the on-screen clock unfalsifiable against a stopwatch. Fix: snap the frame delay to the 10 ms grid first and run all timing math at the *actual* rate.

- **Wagon-wheel aliasing.** At 33 ms of sim per frame, a 200 rad/s wheel sweeps 6.6 rad — more than a full turn — *between* frames. The spoke's on-screen position is then noise: the wheel looks choppy, slow, even reversed, precisely *because* it is fast. Any $|\dot{\phi}| > \pi \cdot \text{fps}$ aliases; no watchable frame rate can honestly depict 30 rev/s. Cure: draw the per-frame sweep as a smear arc and read the numeric $\dot{\phi}$ — never judge speed by the spoke.
- **Player stutter.** A 3,000-frame, 30 MB GIF exceeds what browser decoders sustain; frames render late, adding fake choppiness on top of everything. Long runs belong in mp4.

The moral: validate the instrument before the experiment. When a plot surprises you, the first question is not "what did the physics do?" but "what did the pipeline between the physics and my eyes do?" — same spirit as §9.5, applied to the renderer.

✓ CHECKPOINT – THREE STANDING HABITS FROM THIS SECTION

1. Before blaming a controller for a standing offset, compute F_c/k_p and check whether the arm is simply parked inside the dead-band.
2. After any balancing run, reconcile the wheel's final speed against $K_g \int \sin \psi \, dt$ (equation 25). If the ledger doesn't balance, something in the sim is leaking momentum.
3. When a plot or animation surprises you, audit the instrument first: frame timing, sampling rate vs. the fastest motion (aliasing), and the player itself.

RWP course notes · rev 4 · August 2026 – added §14 (pathology reports from the sim lab: Coulomb dead-band, the wheel-momentum ledger, instrument artifacts). Conventions fixed in §3.1; §14 uses the from-upright ψ notation while the student's code measures θ from down – same physics, mirrored bookkeeping.

The pact stands: derivations and code are the student's; grading and the Phase-8 visualization are the instructor's.